



Desarrollo App Móviles II

Semana 5

Índice

1. ¿Qué es un Memory Leak?	3
2. Herramientas Utilizadas	3
3. Diagnóstico inicial de uso de memoria (Android profiler).....	3
4. Detección de fugas con LeakCanary.....	4
5. Corrección de malas prácticas y mejoras preventivas	5
6. Evidencias.....	6
7. Reflexión final	11

1. ¿Qué es un Memory Leak?

Un *memory leak* (fuga de memoria) ocurre cuando una aplicación mantiene referencias a objetos que ya no son necesarios, impidiendo que el *Garbage Collector* libere la memoria asociada. En Android, este tipo de problema suele originarse por un uso incorrecto del Context, referencias estáticas a vistas o actividades, listeners no liberados o ciclos de vida mal gestionados. Las fugas de memoria pueden provocar un consumo progresivo de RAM, disminución del rendimiento y, en casos extremos, cierres inesperados de la aplicación (*OutOfMemoryError*).

2. Herramientas Utilizadas

Para el análisis y validación del uso de memoria se utilizaron las siguientes herramientas:

- **Android Profiler (Live Telemetry):** Permite monitorear en tiempo real el uso de CPU, memoria (heap) e hilos de ejecución mientras la aplicación se encuentra en funcionamiento.
- **LeakCanary:** Herramienta especializada en la detección automática de fugas de memoria, que analiza objetos retenidos en memoria tras la destrucción de componentes como *Activities* o *ViewModels*.

3. Diagnóstico inicial de uso de memoria (Android Profiler)

Se ejecutó la aplicación en Android Studio utilizando **Android Profiler (Live Telemetry)**, analizando flujos funcionales representativos como:

- Inicio de la aplicación
- Navegación entre pantallas de autenticación
- Escritura y lectura de datos mediante Room
- Permanencia prolongada en la actividad principal

Resultados del análisis:

- **CPU:** Se observaron picos de actividad menores durante operaciones de escritura en la base de datos Room. Durante la mayor parte del tiempo, el uso de CPU se mantuvo bajo, lo que indica que las corutinas están gestionando correctamente la suspensión y reanudación de tareas, evitando el uso innecesario del procesador.
- **Memoria (RAM):** El consumo de memoria se mantuvo estable, con valores aproximados entre 148 MB y 167 MB, sin evidenciar incrementos progresivos ni comportamientos anómalos al navegar repetidamente entre pantallas.
- **Hilos (Threads):** Se verificó que las tareas de mayor carga se ejecutan en hilos secundarios (`Dispatchers.Default`), manteniendo el hilo principal libre y asegurando una experiencia de usuario fluida.

Este diagnóstico inicial no evidenció fugas de memoria activas, pero permitió confirmar un comportamiento estable de la aplicación y sentar la base para un análisis preventivo más profundo.

4. Detección de fugas con LeakCanary

Se integró LeakCanary versión 2.14 en el proyecto mediante la dependencia `debugImplementation`, activándolo únicamente en el entorno de depuración. Durante la ejecución de la aplicación se forzó la creación y destrucción de actividades, así como la navegación repetida entre pantallas, con el objetivo de detectar posibles objetos retenidos en memoria.

Resultados obtenidos:

- LeakCanary reportó “0 Distinct Leaks”.
- No se encontraron objetos retenidos ni fugas nuevas.
- El Logcat confirmó el correcto funcionamiento de la herramienta mediante el mensaje: “LeakCanary is running and ready to detect memory leaks.”

Este resultado confirma que los componentes analizados están siendo liberados correctamente tras completar su ciclo de vida.

5. Corrección de malas prácticas y mejoras preventivas

- **Corrección Realizada: Optimización de referencias de Contexto y control de ciclos en hilos de fondo.**

Justificación:

1. Uso de ApplicationContext: Se modificó la inicialización del repositorio para utilizar `context.applicationContext`. Esto evita que el Singleton retenga referencias a una Activity específica (Leak de Contexto), asegurando que el Recolector de Basura pueda liberar la memoria de las pantallas cuando no están en uso.

Antes

```
1 Usage
85  override fun init(context: Context) {
86      if (isInitialized) return
87
88      prefs = context.getSharedPreferences( name = "veterinaria_prefs", mode = Context.MODE_PRIVATE)
89      database = VeterinariaDatabase.getDatabase(context)
90
91      val ultimoGuardado = prefs?.getString( key = "ULTIMO_TIPO", defValue = null)
92      if (ultimoGuardado != null) {
93          _ultimaAtencionTipo.value = ultimoGuardado
94      }
95  }
```

Después

```
1 Usage
88  override fun init(context: Context) {
89      if (isInitialized) return
90
91      val appContext = context.applicationContext
92      prefs = appContext.getSharedPreferences( name = "veterinaria_prefs", mode = Context.MODE_PRIVATE)
93      database = VeterinariaDatabase.getDatabase(appContext)
94
95      val ultimoGuardado = prefs?.getString( key = "ULTIMO_TIPO", defValue = null)
96      if (ultimoGuardado != null) {
97          _ultimaAtencionTipo.value = ultimoGuardado
98      }
99  }
```

2.Control de Corrutinas: "Se optimizó el algoritmo de búsqueda de citas para que, en lugar de iterar minuto a minuto (lo cual consumía ciclos de CPU y retenía memoria de forma indefinida en horarios no hábiles), el sistema realice saltos lógicos directamente al siguiente bloque disponible. Esto evita el bloqueo de hilos en el CoroutineScope y asegura que la memoria del proceso se libere inmediatamente al encontrar el resultado."

```
37
38     if (esFinDeSemana || fueraDeHorario) {
39         fecha = if (fecha.hour >= 18) {
40             fecha.plusDays( days = 1).withHour( hour = 9).withMinute( minute = 0)
41         } else if (fecha.hour < 9) {
42             fecha.withHour( hour = 9).withMinute( minute = 0)
43         } else {
44             fecha.plusHours( hours = 1).withMinute( minute = 0)
45         }
46         continue
47     }
```

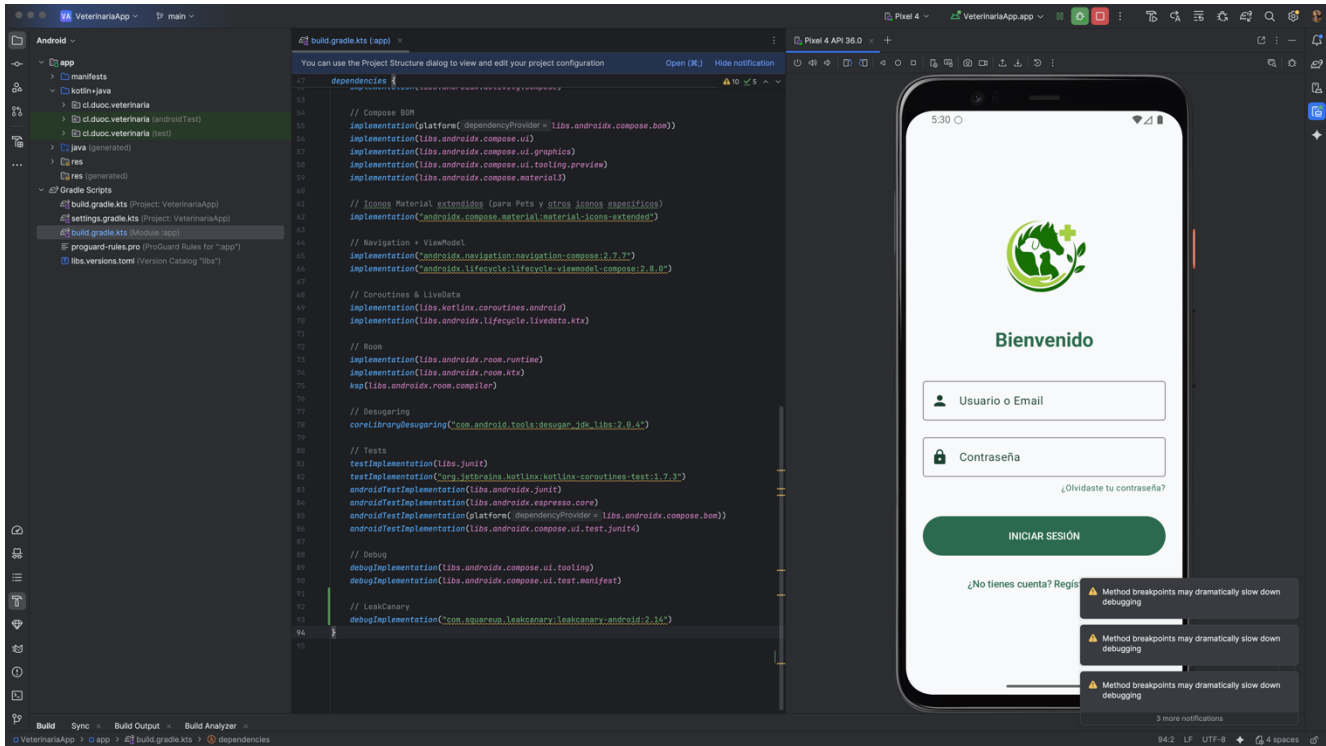
Aunque las herramientas no detectaron fugas en el flujo básico, identifiqué un riesgo latente en la gestión de Contextos del Repositorio y un uso ineficiente de recursos en el servicio de Agenda, procediendo a su corrección preventiva.

6. Evidencias

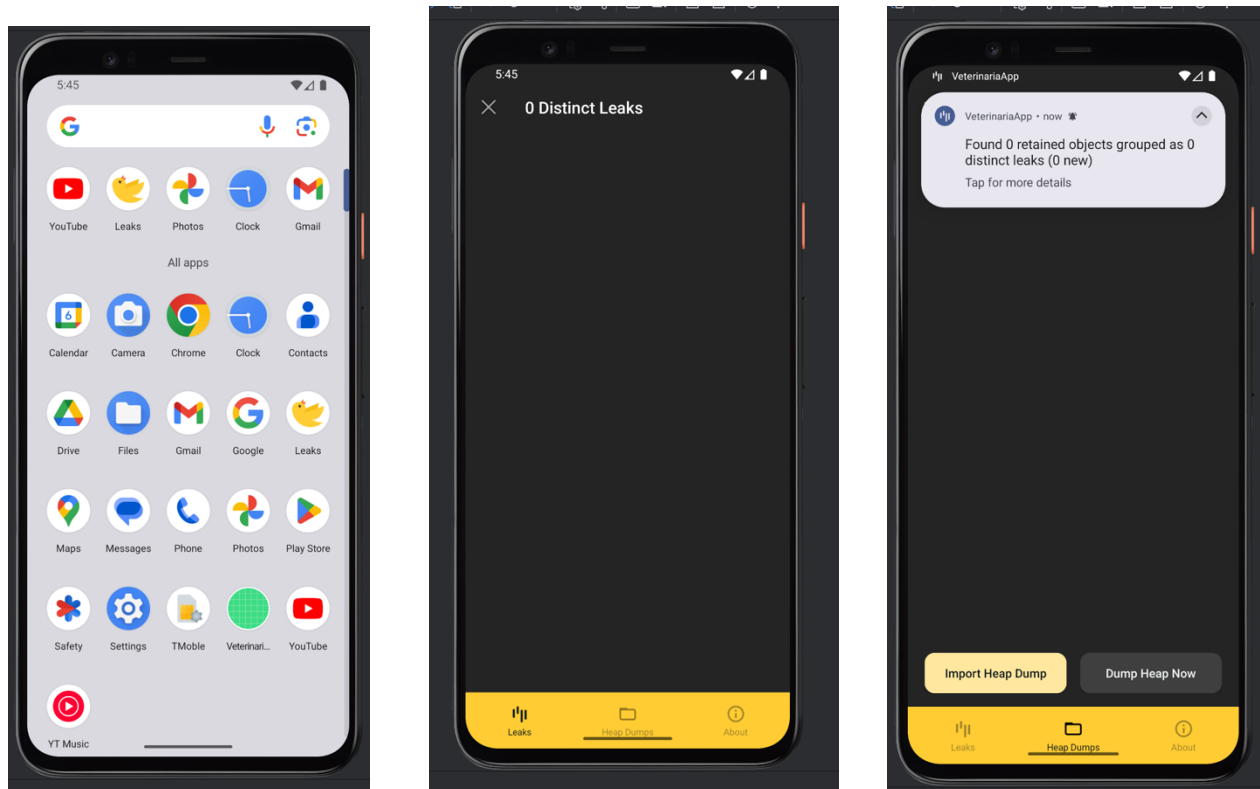
- Dependencias: debugImplementation de LeakCanary en el build.gradle.kts.

```
91
92     // lake canary
93     debugImplementation ("com.squareup.leakcanary:leakcanary-android:2.12")
94 }
95
```

- Implementación en IDE: Android Studio con el código, el emulador al lado.



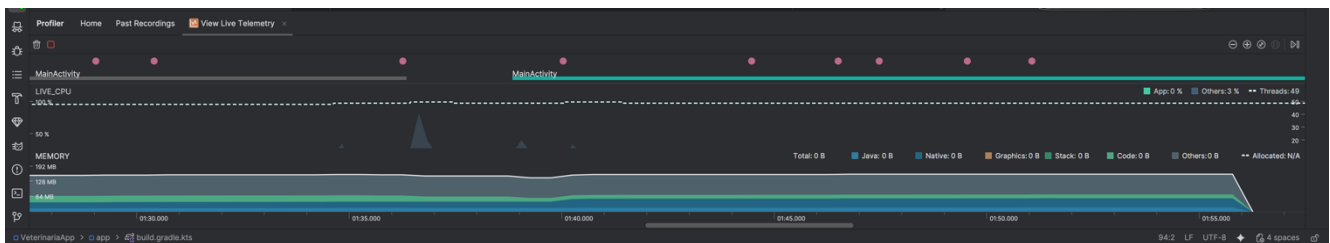
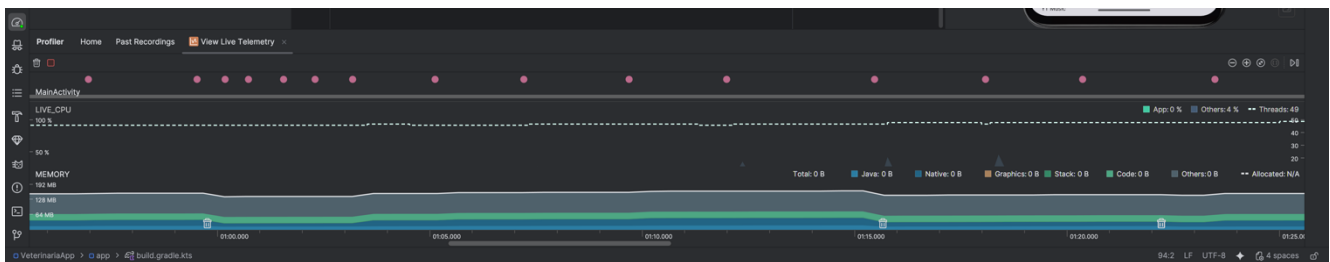
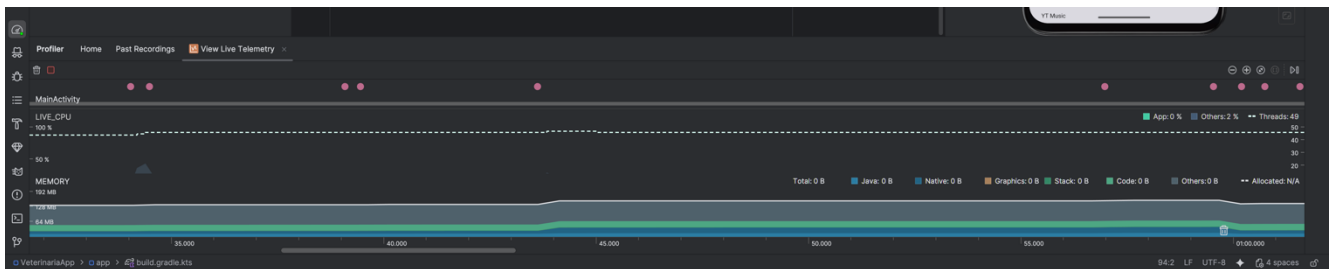
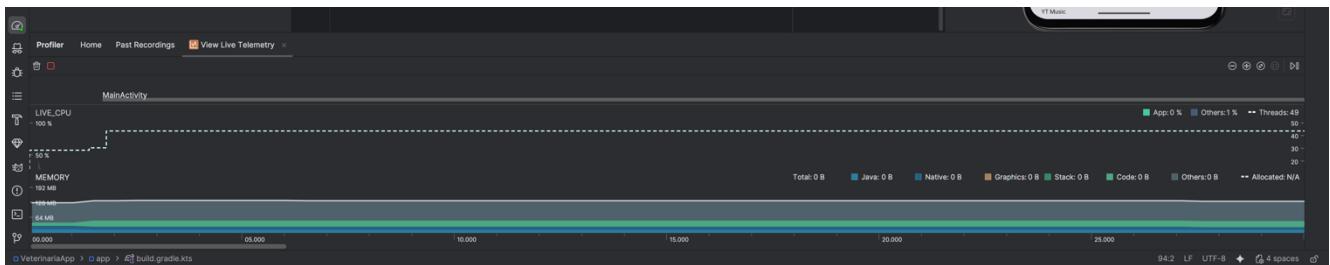
- LeakCanary: emulador "0 Distinct Leaks".



- Logcat: log "LeakCanary is running...".

```
----- PROCESS STARTED (12751) for package cl.duoc.veterinaria -----
2026-02-06 17:48:22.776 12751-12751 uoc.veterinaria cl.duoc.veterinaria I hiddenapi: Accessing hidden field Landroid/app/ActivityThread;:>mH:Landroid/app/ActivityThread$M; (runtime_flags=0, domain=platform, api=unsupported) from LeakCanary/Service
2026-02-06 17:48:22.776 12751-12751 uoc.veterinaria cl.duoc.veterinaria I hiddenapi: Accessing hidden method Landroid/app/ActivityThread;:>currentActivityThread:Landroid/app/ActivityThread; (runtime_flags=0, domain=platform, api=unsupported) from LeakCanary/Service
2026-02-06 17:48:22.776 12751-12751 uoc.veterinaria cl.duoc.veterinaria I hiddenapi: Accessing hidden field Landroid/os/Handler;:>mCallback:Landroid/os/Handler$Callback; (runtime_flags=0, domain=platform, api=unsupported) from LeakCanary/Service
2026-02-06 17:48:22.776 12751-12751 uoc.veterinaria cl.duoc.veterinaria I hiddenapi: Accessing hidden field Landroid/util/Singleton;:>getInstance:Ljava/lang/Object; (runtime_flags=0, domain=platform, api=unsupported) from LeakCanary/Service
2026-02-06 17:48:22.776 12751-12751 uoc.veterinaria cl.duoc.veterinaria I hiddenapi: Accessing hidden method Landroid/util/Singleton;:>get(Ljava/lang/Object; (runtime_flags=0, domain=platform, api=unsupported) from LeakCanary/Service
2026-02-06 17:48:22.827 12751-12766 LeakCanary cl.duoc.veterinaria D LeakCanary is running and ready to detect memory leaks.
```

- Profiler: Gráficas CPU/Memory



1. Monitoreo de Memoria (Sección MEMORY)

Estabilidad del "Heap": Lo más importante que se observa son las líneas horizontales casi planas. Esto indica que la aplicación tiene un consumo de memoria saludable. Si hubiera una fuga (leak), se vería que la línea verde/azul sube constantemente como una rampa sin bajar nunca.

Eventos de Garbage Collection (GC): En la parte inferior de la gráfica de memoria se ven unos pequeños íconos de "basurero". Esto indica que el sistema Android entró a limpiar la memoria. Como el nivel de memoria se mantiene estable después de estos íconos, se confirma que los objetos se están liberando correctamente.

2. Monitoreo de CPU (Sección LIVE_CPU)

Picos de Actividad: Los triángulos grises que aparecen en la línea de tiempo coinciden con los puntos rosados de la parte superior (interacciones del usuario). Esto significa que la CPU solo trabaja cuando se toca la pantalla o se hace algo, lo cual es eficiente.

Uso en Reposo: Cuando no se toca la app, el uso de CPU baja a casi 0%, lo que demuestra que no hay procesos "hambrientos" de recursos corriendo infinitamente en segundo plano.

3. Eventos de Usuario (Puntos Rosados y Barras Azules)

MainActivity: La barra azul horizontal etiquetada como MainActivity muestra el ciclo de vida de la pantalla principal. Al navegar o rotar, el Profiler registra si la actividad se destruye y se recrea correctamente.

Interacciones: Cada punto rosado es un "tap" o gesto que se realizó. El hecho de que la memoria no suba drásticamente después de muchos toques es una prueba visual de que la gestión de estados (con ViewModels) es correcta.

- Analyze Memory Usage (Heap Dump)

Class Name	Allocations	Native Size	Shallow Size	Retained Size
app heap	67,504	1,083,712	3,797,098	2,999,676
EmojiCompat (androidx.emoji2.text)	2	0	248	352,479
Bitmap (android.graphics)	3	1,008,715	174	128,491
ZoneRulesProvider (java.time.zone)	1	0	124	60,482
LoadersApp (vet.vetapp)	3	0	381	35,999
DesPathList (daxiv.system)	1	0	32	19,012
AndroidComposeView (androidx.compose.ui.platform)	2	0	1,277	11,663
AndroidViewHolder (androidx.compose.ui.viewinterop)	1	0	128	10,160
ClassReference (java.lang.reflect)	2	0	156	10,108
AndroidViewHolder (androidx.compose.ui.platform)	1	0	120	9,896

Análisis del Heap Dump

- **Detección de Fugas (Leaks: 0):** En la esquina superior izquierda, se observa el contador de "Leaks: 0". Esta es la validación definitiva de que, tras completar todo el flujo de la app, el motor de análisis de Android Studio no encontró ningún objeto (Activity, Fragment o Context) que haya quedado retenido ilegalmente.
- **Contador de Clases y Objetos:** Se muestran 7,152 clases cargadas y más de 67,000 objetos (Allocations). El hecho de que no haya fugas entre tal cantidad de elementos demuestra una arquitectura sólida.
- **Retained Size (Memoria Retenida):** El valor de 2,999,676 (aprox. 3MB) en la columna "Retained Size" para el bloque principal muestra cuánta memoria se liberaría si se eliminaran esos objetos. Es un valor muy bajo y saludable para una app de este tipo.
- **Análisis de Bitmaps:** Se observa que la clase android.graphics.Bitmap tiene un peso importante en la memoria nativa. Esto es normal y esperado, ya que corresponde a los recursos gráficos, iconos y el logo de la veterinaria que se cargó en la pantalla de bienvenida y registro.

8. Reflexión final

La realización de esta actividad permite concluir que la gestión de memoria no es solo una tarea técnica, sino un pilar fundamental de la experiencia de usuario. El uso de herramientas como Android Profiler y LeakCanary nos otorga una visibilidad crítica sobre el comportamiento interno de la aplicación, permitiendo transformar fallos invisibles en soluciones tangibles. Más allá de corregir errores detectados, la importancia de esta práctica radica en la prevención proactiva: identificar riesgos arquitectónicos (como el manejo de contextos en Singletons) antes de que afecten al usuario final. Un desarrollo de alta calidad no solo es aquel que funciona, sino aquel que respeta y optimiza los recursos limitados del dispositivo, asegurando la estabilidad y escalabilidad del proyecto a largo plazo.



Reservados todos los derechos Fundación Instituto Profesional Duoc UC. No se permite copiar, reproducir, reeditar, descargar, publicar, emitir, difundir, de forma total o parcial la presente obra, ni su incorporación a un sistema informático, ni su transmisión en cualquier forma o por cualquier medio (electrónico, mecánico, fotocopia, grabación u otros) sin autorización previa y por escrito de Fundación Instituto Profesional Duoc UC. La infracción de dichos derechos puede constituir un delito contra la propiedad intelectual.